Deployment Plan

<APP_NAME> = Liquor Counter POS & Inventory Platform — build to live hosting

Document	Production deployment plan and release runbook
Version	1.0 (draft for review) — 2026-11-13
Release	<RELEASE_VERSION> (git tag `v<MAJOR.MINOR.PATCH>`)
Audience	Engineering, QA, Operations
Owner / approver	<RELEASE_MANAGER> / <ENGINEERING_LEAD>
Basis	Written from this repository (`package.json`, `.env.example`, `src/app.ts`, `drizzle/`, `AGENTS.md`). Nothing here has been executed against a live environment.

Before you use this plan — information to supply. Every `<PLACEHOLDER>` below is a value this repository cannot tell us. Fill these in first:

Placeholder	What to provide
`<HOSTING_PROVIDER>`, `<REGION>`	Where the app runs (VM, container service, Kubernetes) and its region. The database in `.env` is hosted Postgres in `ap-southeast-1`; run the app in the same region (§10.3).
`<APP_DOMAIN>`, `<DNS_PROVIDER>`, `<CDN>`	Public hostname (e.g. `pos.example.com`), DNS host and CDN, if any.
`<DB_HOST>`, `<DB_NAME>`, `<DB_ADMIN_URL>`	Production Postgres endpoint, database name, and a *direct* (non-pooled) admin connection for migrations.
`<REGISTRY>`, `<CI_SYSTEM>`	Container/artifact registry and CI platform (none is configured in the repo today).
`<VAULT>`	Secrets manager in use (e.g. cloud secret store, Vault, 1Password).
`<SMTP_HOST>`, `<RAZORPAY_*>`	Mail relay and live payment-gateway credentials / webhook.
`<RPO>` / `<RTO>`, `<RETENTION_POLICY>`	Recovery objectives and record-retention period (GST / excise).
`<WINDOW>`	Approved release window — outside trading hours of the outlets.
Contact names / on-call	For the matrix in Appendix C.

Conventions: GAP marks something the code or repo does *not* provide today and must be added or accepted as a risk before go-live (collected in §1.4).

1. Overview & Scope

1.1 Purpose of the release

Take a tested build of `<APP_NAME>` — a point-of-sale, inventory and accounting platform for liquor retail stores and bars in India (single stores or multi-store syndicates) — from source control to live hosting, with a database change path, verification and a tested way back. Optional release items: the database index/constraint package in `docs/db-optimization/` (separately gated, §4.7).

1.2 System components

Component	Detail
API + web server	Node.js ≥ 22, Fastify 5 (`@fastify/jwt`, `helmet`, `cors`, `rate-limit`, `multipart`). One process serves `/api/v1/*`, `/health` **and** the built React app at `/` (any unknown path falls through to the SPA). Listens on `0.0.0.0:$PORT` (default 3001).
Frontend	React + Vite, built into `dist/frontend` by `npm run build:frontend` and served by the API. No separate web tier is required.
Database	PostgreSQL via Drizzle ORM. 64 tables, 25 SQL migrations in `drizzle/`. Hosted Postgres (pooled endpoint for the app, direct endpoint for DDL).
Background work	In-process: daily billing/dunning job (`BILLING_JOB_ENABLED`), print-queue sender, invoice OCR (CPU/memory heavy). Redis (`REDIS_URL`) is listed in `.env.example` as "Phase 2" and is **not required** today.
Integrations	Razorpay (payments + webhook `/api/v1/webhooks/razorpay`), SMTP e-mail, branch **print agent** (local program on the shop PC that polls the API), ESC/POS network printers.
Clients	Browsers on shop counters (POS works offline-queued), phones/tablets for stewards.

1.3 Target environments and assumptions

Env	Purpose	Data	Database
dev	Developer machines (`npm run dev:local`)	Seeded demo data	Embedded PGlite (`pglite://./.pglite`)
ci	Automated tests	Synthetic	In-memory PGlite
staging	Production-like rehearsal, UAT, migration dry-run	Restored copy of production (masked) or realistic seed	Separate Postgres, same version/region as prod
prod	Live customers	Real	Production Postgres with PITR enabled

Assumptions: (a) single region, one app deployment with ≥ 2 instances behind a load balancer; (b) release only in `<WINDOW>`; (c) database migrations are **forward-only and additive** within a release (Drizzle generates no "down" scripts); (d) the app is stateless apart from two in-memory throttles (§5.4); (e) every shop is reachable only over the internet, so network printers are driven through the print agent.

1.4 Known gaps to close or accept before go-live

#	Gap found in the repository	Consequence / action
G1	GAP No Dockerfile, CI workflow, or IaC in the repo.	Sections 3 and 5 give proposed files; they must be created and reviewed (owner: DevOps).
G2	GAP `/health` only proves the process is up; it does not touch the database.	Use it as the *liveness* probe only. Add a readiness check (`select 1`) or accept the synthetic login check in §8 as the DB signal.
G3	GAP No SIGTERM/graceful-shutdown handler in `src/index.ts`.	In-flight bills can be cut during a rolling restart. Add a handler that calls `app.close()`, and set a pre-stop delay and grace period (§5.3).
G4	Existing tests are not green on the current tree: **12 integration** and **8 e2e** failures pre-date this plan (stale selectors/wording, one needs `ARCHIVE_RETENTION_DAYS=0`).	Triage before gating CI. Until then the gate is "**no new failures versus the recorded baseline**" (§3.3).
G5	Migrations `0017` – `0024` and `drizzle/meta/_journal.json` were **uncommitted** when this plan was written.	Commit them before tagging. A database missing them fails every bill (AGENTS.md §8).
G6	Login throttle and rate limits are **per instance** (in-memory).	With N instances the effective limit is N× higher. Accept, or add Redis later (§5.4).

G7	`npm run db:migrate` runs through `tsx` (a dev dependency) and reads `./drizzle` from source.	Run migrations from a release job that has dev dependencies, **not** from the slim runtime image (§4.3).

2. Prerequisites & Access

2.1 Tools

Tool	Version / use
Node.js / npm	Node 22 LTS (`engines: >=22`), npm 10+
git	Tagging and release branches
PostgreSQL client	`psql` / `pg_dump` / `pg_restore` 16+ (≥ server major)
Docker (or buildah)	Image build; `<REGISTRY>` push access
kubectl / cloud CLI	Only if deploying to Kubernetes / a cloud container service
Playwright + Chromium	e2e tests (`npx playwright install chromium`)
Scanners	`npm audit`, Trivy (image), gitleaks (secrets) — §7.3

2.2 Accounts, credentials and secrets management

- All secrets live in `<VAULT>`; **never** in git, images, CI logs or chat. `.env` is a developer convenience only — do not bake it into an image (`npm start` reads `--env-file=.env`; in production start with `node dist/index.js` and inject variables from the platform).
- Secrets to manage: `DATABASE_URL`, `JWT_SECRET` (≥ 32 chars, enforced), `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `SMTP_PASS`, registry token, DB admin URL, backup-bucket key.
- Rotate on: staff departure, suspected leak, and every `<ROTATION_DAYS>` days. Rotating `JWT_SECRET` signs every user out — do it in a window.

2.3 Access matrix (least privilege)

Role	Git	CI	Registry	Vault	Staging	Prod app	Prod DB
Developer	write (PR)	run	pull	dev only	deploy	none	none
QA	read	view	—	—	test	smoke only	none
Release manager	tag	approve	push	read	deploy	approve	—
DevOps / SRE	read	admin	admin	read/write	admin	deploy	migrate (time-boxed)
DBA	read	—	—	DB secrets	admin	—	admin
Support	—	—	—	—	—	read logs	none

Production database admin rights are granted just-in-time for the release and revoked afterwards. Two-person approval for any production DDL.

3. Build Process

3.1 Branching, tagging and versioning

- **Trunk-based:** short-lived feature branches → PR → `main` (protected: required review + green CI). Commit style: conventional commits (`feat(scope):`, `fix:`, `chore(db):`).
- **Release:** cut `release/x.y` from `main`; only fixes land there. Tag `vX.Y.Z` on the release commit (annotated, signed if possible).
- **Versioning:** SemVer. MAJOR = breaking API/contract or non-additive schema change; MINOR = backward-compatible feature (including additive migrations); PATCH = fix. The image tag is the git tag plus the short SHA: `<REGISTRY>/<APP_NAME>:1.4.0` and `:1.4.0-a1b2c3d`. **Never deploy `latest`.** Align `package.json` version (currently `0.1.0`) with the tag.

3.2 CI pipeline (proposed — GAP G1)

#	Stage	Command	Gate
1	Install	`npm ci && npm ci --prefix frontend`	lockfiles unchanged
2	Lint / types	`npm --prefix frontend run lint` (oxlint); `npx tsc --noEmit`	zero errors

3	Unit	`npm run test:unit`	all pass
4	Integration	`npm run test:integration` (real app on in-memory PGlite)	no new failures vs baseline (G4)
5	Build	`npm run build` (= frontend build + `tsc → dist/`)	exit 0
6	E2E	`npm run test:e2e` (needs the frontend build; boots its own PGlite on :3001 — never run where a dev server holds :3001)	no new failures vs baseline
7	Security scans	`npm audit --omit=dev --audit-level=high`; gitleaks; Trivy on the image	no High/Critical without waiver
8	Package	`docker build -t <REGISTRY>/<APP_NAME>:$VER .`	image builds
9	Publish	`docker push`; attach SBOM and test reports	digest recorded in the release ticket

Build note. `npm start` has a `prestart` hook that rebuilds. In production never build at start: build once in CI and run `node dist/index.js`. Build the image `FROM node:22-bookworm-slim` (glibc; the `@napi-rs/canvas` native module used by invoice OCR does not suit Alpine).

3.3 Test baseline

At the time of writing: integration **12 failed / 392 passed**, e2e **8 failed / 67 passed** on an unmodified tree. Record the failing test names in the release ticket; a release is blocked by *any failure not in that list*.

3.4 Artifact and storage

Artifact	Contents	Storage / retention
Container image	`dist/` (API + `dist/frontend`), production `node_modules`, **and** the `drizzle/` folder	`<REGISTRY>`, immutable tags, keep last 20 releases + all tagged
Migration bundle	`drizzle/*.sql` + `_journal.json` at the tag (also the release job image)	same registry / release assets
Evidence	Test reports, SBOM, scan results, image digest	CI artifact store, 12 months

4. Database Setup & Migration

4.1 Provisioning (first install only)

```
# as DBA, on the DIRECT endpoint
createdb -h <DB_HOST> -U <DB_ADMIN> <DB_NAME>
psql "<DB_ADMIN_URL>" -c "CREATE ROLE <APP_DB_USER> LOGIN PASSWORD '<from vault>';"
psql "<DB_ADMIN_URL>" -c "GRANT CONNECT ON DATABASE <DB_NAME> TO <APP_DB_USER>;"
# app user owns the objects it migrates (Drizzle creates tables, enums, indexes)
psql "<DB_ADMIN_URL>" -c "ALTER DATABASE <DB_NAME> OWNER TO <APP_DB_USER>;"
```

Enable **point-in-time recovery** and note its window. Use the **pooled** URL in `DATABASE_URL` for the running app and the **direct** URL for migrations and backups.

4.2 Migration scripts and execution order

Migrations are the ordered SQL files `drizzle/0000_… → 0024_…` listed in `drizzle/meta/_journal.json`; Drizzle applies the ones not yet recorded in `drizzle.__drizzle_migrations`, in journal order, inside **one transaction** — a failure rolls the whole pending batch back (which is also why `CREATE INDEX CONCURRENTLY` cannot go through it). **Order of operations for every release: migrate first, then deploy the app** — a missing column fails every bill because the counter-sheet handler runs inside the bill transaction.

Compatibility rule: release N's schema must work with release N-1's code (add columns/indexes/tables; never drop or rename in the same release — use expand → migrate → contract over two releases). This is what makes an application-only rollback safe (§11).

4.3 Running the migration

```
# release job (has devDependencies + the repo at the release tag), DIRECT url, from a bastion/CI runner
git checkout v<RELEASE_VERSION>
npm ci
export DATABASE_URL="<DB_ADMIN_URL>"          # direct, not -pooler
npm run db:migrate                            # prints "Migrations applied"

# confirm
psql "$DATABASE_URL" -tc "select count(*) from drizzle.__drizzle_migrations;"   # expect 25 + new ones
```

4.4 Backup before migration (mandatory gate)

```
# 1) provider snapshot / branch + note the timestamp (preferred, instant)
# 2) logical dump as a second copy, stored outside the DB account, encrypted
pg_dump -Fc -h <DB_HOST> -U <DB_ADMIN> -d <DB_NAME> -f pre_<RELEASE_VERSION>_$(date +%F_%H%M).dump
pg_restore --list pre_<RELEASE_VERSION>_*.dump | head      # proves the file is readable
sha256sum pre_<RELEASE_VERSION>_*.dump > pre_<RELEASE_VERSION>.sha256
```

Restore rehearsal: restore the dump to staging at least once per quarter and before any first release.

4.5 Seed data

- **Do not run `npm run db:seed` in production.** It creates demo users with a known password and refuses to run when `NODE_ENV=production` unless `ALLOW_PROD_SEED=1` — leave that unset.
- Create the first Super Admin (the software provider's platform owner) on the server against the production URL: `npm run super-admin` (see `src/db/create-super-admin.ts`); store the generated credentials in the vault and force a password change.
- Reference data (categories, GST rates, sale types) is loaded by the onboarding workflow / agreed import — `<REFERENCE_DATA_SOURCE>`.

4.6 Validation after migration (read-only SQL)

```
-- run the package from docs/db-optimization/sql/tests/
psql "$DATABASE_URL" -f t02_business_invariants.sql   -- every violations = 0
psql "$DATABASE_URL" -f t03_referential_integrity.sql -- must return no rows
psql "$DATABASE_URL" -f t01_fingerprint.sql > fp_after.txt   -- compare with fp_before.txt for DDL-only releases
```

4.6a Rollback scripts

Drizzle migrations have no automatic down step. Rollback options, in order of preference: (1) **leave the additive schema in place** and roll back only the application; (2) run the release's hand-written `*_down.sql` (the `docs/db-optimization` phases ship tested ones); (3) restore the §4.4 backup / PITR. Every migration PR must state which of the three applies.

4.7 Optional release item — index & constraint package

`docs/db-optimization/` contains Phase 1 (41 indexes, built `CONCURRENTLY`, no data change) and Phase 2 (11 CHECKs + 2 foreign keys, `NOT VALID` then validated, pre-check required). Deploy it as its own change after the application release is stable; follow its `08-deployment-runbook.md`. Run the `_concurrent` files with `psql` on the direct connection — they cannot go through the migrator or a transaction pooler.

5. Server & Infrastructure Deployment

5.1 Environment configuration

Variable	staging	prod	Notes
`NODE_ENV`	production	production	Enables the safety checks below; staging must behave like prod.
`PORT`	3001	3001	Listens on 0.0.0.0.
`DATABASE_URL`	<STG_POOLED_URL>	<PROD_POOLED_URL>	Pooled endpoint; `?sslmode=require`. From vault.
`JWT_SECRET`	unique per env	unique, ≥ 32 chars	**App refuses to start** if missing/short in production.
`JWT_EXPIRES_IN`	8h	8h	Session length.
`CORS_ORIGIN`	https://<STG_DOMAIN>	https://<APP_DOMAIN>	**Required in production, not "*"** — app refuses to start. Same origin as the SPA.

`TRUST_PROXY`	1	number of proxy hops (e.g. 2 with CDN + LB)	Needed so rate limits and login lockout see the real client IP.
`APP_URL`	https://<STG_DOMAIN>	https://<APP_DOMAIN>	Used in e-mail links.
`PAYMENT_PROVIDER`	mock + `ALLOW_MOCK_PAYMENTS=1` + `MOCK_PAYMENT_SECRET`	**razorpay**	Prod **refuses to start** on the built-in test gateway. Needs `RAZORPAY_KEY_ID/_SECRET/_WEBHOOK_SECRET`.
`SMTP_HOST/PORT/SECURE/USER/PASS`, `MAIL_FROM`	test relay	<SMTP_*>	Empty host = e-mails are only stored in the outbox, never sent.
`BILLING_JOB_ENABLED`	0	unset/1 on **one** instance, **0** on the others	Billing/dunning timer (every 6 h, in-process, **on by default**). See §5.4.
`BUSINESS_UTC_OFFSET_MINUTES`	330	330	"Business day" boundary (IST). Do not change after go-live.
`ALLOW_PROD_SEED`, `ARCHIVE_RETENTION_DAYS`, `DB_LOG_QUERIES`	unset	**unset**	Test/diagnostic switches; must be unset in prod.
`DB_DNS_FALLBACK`	1	1	Public-DNS fallback if the resolver cannot find the DB host.

5.2 Container image (proposed Dockerfile)

```
FROM node:22-bookworm-slim AS build
WORKDIR /app
COPY package*.json ./
COPY frontend/package*.json frontend/
RUN npm ci && npm ci --prefix frontend
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:22-bookworm-slim
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/package.json ./package.json
COPY --from=build /app/drizzle ./drizzle
USER node
EXPOSE 3001
HEALTHCHECK --interval=15s --timeout=3s CMD node -e "fetch('http://127.0.0.1:3001/health').then(r=>process.exit(r.ok?0:1),
()=>process.exit(1))"
CMD ["node","dist/index.js"]
```

5.3 Kubernetes (or equivalent container service) — minimal manifest

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: <APP_NAME> }
spec:
  replicas: 2
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  template:
    spec:
      terminationGracePeriodSeconds: 30
      containers:
      - name: app
        image: <REGISTRY>/<APP_NAME>:<RELEASE_VERSION>
        ports: [{ containerPort: 3001 }]
        envFrom: [{ secretRef: { name: <APP_NAME>-env } }]     # synced from <VAULT>
        resources: { requests: { cpu: 250m, memory: 512Mi }, limits: { cpu: "1", memory: 1Gi } }
        readinessProbe: { httpGet: { path: /health, port: 3001 }, periodSeconds: 5 }
        livenessProbe:  { httpGet: { path: /health, port: 3001 }, periodSeconds: 15, failureThreshold: 3 }
        lifecycle: { preStop: { exec: { command: ["sleep","10"] } } }   # drain before SIGTERM (see G3)
```

On a plain VM use the same image under systemd/Compose with `restart: always` and two instances behind the load balancer.

5.4 Scaling and statefulness

- **CPU-bound work:** password hashing (bcrypt) on login and invoice OCR. Start at 2 replicas × 1 vCPU / 1 GiB; horizontal autoscaler on CPU 65 % (min 2, max `<MAX_REPLICAS>`). Uploads are limited by `MAX_UPLOAD_BYTES` (`invoice.routes.ts`); allow that body size at the load balancer.
- **No sticky sessions needed** (JWT bearer tokens).
- **Per-instance state (G6):** login throttle and rate limits are in memory — limits multiply by replica count. The billing/dunning timer (every 6 h) also runs inside **every** process unless `BILLING_JOB_ENABLED=0`. Reminders are guarded by a unique index (`uq_dunning_once`), so duplicates are prevented, but run it on one instance (set `=0` on the rest) to avoid wasted work and racing invoice generation.
- **DB connections:** pooled endpoint; keep `replicas × pool size` below the pooler's limit. Each cash bill makes ≈ 24 queries, so app and database must be in the same region (§10.3).

5.5 Networking, load balancer and TLS

- **Load balancer** (L7): terminate TLS, HTTP→HTTPS 301, health check `GET /health` (expect 200), idle timeout ≥ 60 s, request body limit ≥ `MAX_UPLOAD_BYTES`, forward `X-Forwarded-For/Proto` and set `TRUST_PROXY` to the hop count.
- **Certificates:** managed certificate (ACM / Let's Encrypt) for `<APP_DOMAIN>`, auto-renew, alert 21 days before expiry. TLS 1.2+ only; HSTS (`max-age ≥ 31536000`) at the edge — verify the header in §10.
- The app server is **not** publicly addressable: only the load balancer may reach port 3001.

6. Hosting & Domain Configuration

6.1 Provider and DNS

Item	Value
Hosting provider / region	<HOSTING_PROVIDER> / <REGION> (co-locate with the database region)
Database	<DB_HOST> (hosted Postgres, pooled + direct endpoints, PITR on)
Registrar / DNS	<DNS_PROVIDER>; TTL lowered to 300 s one day before cut-over

Record	Type	Value	Purpose
`<APP_DOMAIN>`	CNAME / A (alias)	<LB_OR_CDN_HOSTNAME>	The app (UI + API, one origin)
`<STG_DOMAIN>`	CNAME	<STG_LB_HOSTNAME>	Staging
`<APP_DOMAIN>`	CAA	`0 issue "<CA>"`	Restrict who may issue certificates
`<MAIL_DOMAIN>`	TXT (SPF, DKIM, DMARC)	per mail provider	Billing / password-reset e-mail deliverability

6.2 CDN (optional but recommended)

- Origin = the load balancer. **Cache** `/assets/*` and other hashed files from `dist/frontend` (long TTL, `immutable`). **Do not cache** `/api/*` or `/index.html` (short/no-cache), so a release is visible immediately.
- Forward `Authorization` and all methods on `/api/*`; pass client IP in `X-Forwarded-For` (counts as one more proxy hop for `TRUST_PROXY`).
- Enable the CDN's WAF managed rules and bot/rate protection in front of `/api/v1/auth/login`.

6.3 Firewall and security-group rules

Direction	Source → Destination : port	Rule
Inbound	Internet/CDN → Load balancer : 443	Allow
Inbound	Internet → Load balancer : 80	Allow, redirect to 443 only
Inbound	Load balancer → App : 3001	Allow (security-group reference); deny all else
Inbound	Razorpay → `/api/v1/webhooks/razorpay` (443)	Allow; the request is authenticated by the webhook signature (`RAZORPAY_WEBHOOK_SECRET`)
Outbound	App → Database : 5432	Allow to <DB_HOST> only (use the database allow-list / private link if offered)
Outbound	App → SMTP : 587, Razorpay API : 443	Allow to named destinations
Outbound	App → branch printers : 9100	**Block / not used.** A cloud server cannot reach shop LAN printers; set branches to **print via agent** (local print agent polls the API over HTTPS).

Admin	Bastion / CI runner → App hosts : 22, DB : 5432	Allow from fixed IPs / SSM only; no public SSH

7. Security & Compliance

7.1 Controls

Area	Control (and what the code already does)
Secrets	From `<VAULT>` at deploy time; image contains none; CI uses short-lived tokens; secret scanning on every push. The app refuses to start with a missing/short `JWT_SECRET`, a wildcard `CORS_ORIGIN`, or the test payment gateway in production.
HTTPS	Enforced at LB/CDN (301 + HSTS). The API also sends `helmet` security headers. Cookies are not used for auth (bearer JWT) — keep tokens out of URLs and logs.
AuthN / AuthZ	bcrypt password hashes; role + permission checks re-read the user from the DB on every request, so deactivation is immediate; login lockout and per-IP rate limit (needs correct `TRUST_PROXY`).
Access logging	Structured request logs (Fastify/pino JSON on stdout) shipped to the log platform; LB access logs on; admin actions in `audit_log` (Super Admin → Audit Log). **Never log** Authorization headers, passwords, tokens or card data.
Data protection	Encryption in transit (TLS to DB) and at rest (provider default). The database holds customer phone numbers and uploaded supplier-invoice files (`invoice_attachments`): treat dumps/backups as sensitive and encrypt them.
Domain compliance	Bills and journal vouchers are never deleted (void/reverse only) and an audit trail records every stock change — retain per `<RETENTION_POLICY>` (GST / excise rules). Permanent business deletion is possible only for the Super Admin after the archive period (don't set `ARCHIVE_RETENTION_DAYS` in prod).

7.2 Backup policy

Backup	Frequency	Retention	Target
Continuous WAL / PITR	continuous	<PITR_DAYS> (≥ 7)	RPO <RPO>
Logical dump (`pg_dump -Fc`)	daily + before every release	30 d daily, 12 monthly	separate account/bucket, encrypted, write-once
Restore test	quarterly + before first go-live	—	staging; RTO <RTO> measured and recorded

7.3 Vulnerability scanning

- Every build: `npm audit`, Trivy (image + OS packages), gitleaks, dependency-update PRs weekly.
- Before go-live and yearly: external penetration test of the public surface (login, upload, webhook); fix High/Critical before launch.
- Patch SLA: Critical 48 h, High 7 d, Medium 30 d.

8. Monitoring, Logging & Alerting

8.1 Health checks

Check	Endpoint / method	Frequency
Liveness / LB health	`GET /health` → `{"status":"ok"}` (process only — G2)	5–15 s
Uptime (external)	`GET https://<APP_DOMAIN>/api/v1/health` and `GET /` (SPA loads)	1 min, from ≥ 2 locations
Synthetic transaction (the real DB signal)	Log in as a dedicated test cashier and call `GET /api/v1/me` + `GET /api/v1/products`	5 min
Certificate expiry	TLS probe	daily

8.2 Metrics, logs and alert thresholds

Signal	Alert threshold	Severity	Responsible
Uptime / synthetic login fails	2 consecutive failures	P1	On-call SRE
HTTP 5xx rate	> 1 % for 5 min	P1	On-call SRE

Bill-create p95 latency	> <P95_MS> ms for 10 min (baseline in §10)	P2	Backend on-call
Bills created in opening hours	0 for 30 min across all outlets	P1	On-call SRE + Support
CPU / memory per instance	> 80 % for 10 min; OOM kill	P2	SRE
DB: connections, CPU, storage	> 80 % of limit; storage > 80 %	P2	DBA
DB: replication / PITR status; failed backup	any failure	P2	DBA
Print queue (`print_jobs` PENDING)	oldest pending > 10 min	P2	Support / Backend
Payment webhook failures; e-mail `FAILED` in `email_outbox`	> 3 in 15 min	P2	Backend on-call
Auth: login 429 / lockouts	spike > 5× baseline	P3 (possible attack)	Security
Certificate / domain expiry	< 21 days	P3	SRE

Logs: container stdout (JSON) → `<LOG_PLATFORM>`; retain 30 d hot / 12 months archive; index by request id, route, status, branch. For a database round-trip investigation set `DB_LOG_QUERIES=1` briefly on one instance (never left on). **Escalation:** P1 page immediately, P2 within 30 min in hours, P3 next business day — contacts in Appendix C.

9. Deployment Procedure (Release Runbook)

Window: `<WINDOW>` (outside outlet trading hours). Roles: **RM** release manager, **SRE**, **DBA**, **DEV** developer on call, **QA**, **SUP** support. Durations are planning estimates — replace with measured values after the staging rehearsal.

#	Task	Role	Est.	Command / evidence
T-1 day				
1	Go/no-go meeting; confirm test baseline, scan results, change ticket approved	RM	30 m	Ticket updated
2	Rehearse the release on staging with a restored prod copy; time the migration	SRE, DBA, QA	2 h	Staging report
3	Lower DNS TTL to 300 s (only if DNS changes); notify outlets of the window	SRE, SUP	10 m	—
T-0 (release window)				
4	Freeze merges; verify the tag and image digest match the approved build	RM	10 m	`docker manifest inspect …@sha256:…`
5	Open status communication ("maintenance started")	SUP	5 m	§11.4 template
6	**Backup gate:** snapshot + `pg_dump`, verify readable	DBA	15 m	§4.4; record snapshot id
7	Pre-migration validation: `t02` and `t03` clean; save fingerprint	DBA	10 m	§4.6
8	Run the migration (additive) on the direct endpoint	DBA / SRE	5–10 m	`npm run db:migrate`
9	Post-migration validation: migration count, `t02`, `t03`	DBA	10 m	§4.6
10	Set/verify secrets and env (§5.1) in the vault → platform	SRE	10 m	diff against the env table
11	Deploy the new image: rolling update, 1 surge, 0 unavailable	SRE	10 m	`kubectl set image deploy/<APP_NAME> app=…:<RELEASE_VERSION>`; `kubectl rollout status`
12	Confirm all instances ready; check logs for startup errors (config refusals appear here)	SRE, DEV	10 m	`kubectl logs -l app=<APP_NAME> --tail=100`
13	Run the smoke suite on production (§10.1)	QA	30 m	Checklist signed
14	Confirm exactly one instance runs the billing timer (`BILLING_JOB_ENABLED=0` elsewhere); confirm webhook delivery	SRE, DEV	10 m	Razorpay dashboard test event
15	Go / no-go decision; announce "release complete" or invoke rollback (§11)	RM	10 m	Sign-off table §10.4
16	Start the 24–72 h monitoring window; unfreeze merges	RM, SRE	—	§12

Total in-window estimate: about 2.5–3.5 hours including smoke tests, with the rollback decision point no later than step 13.

10. Testing & Verification

10.1 Smoke tests (production, after step 12)

#	Check	Pass criterion
1	`GET /health`, `/api/v1/health`, `GET /` over HTTPS; HTTP redirects to HTTPS; HSTS header present	200 / 301; valid certificate
2	Log in as each role (cashier, manager, owner) with the test accounts; wrong password shows the attempts-left message	Lands on the right dashboard
3	POS: open shift → scan/add item → cash bill → receipt shows; stock decreases by the sold quantity	Bill number issued; one stock-audit row
4	Void that bill (manager, with reason)	Stock restored; reversing voucher posted
5	CREDIT sale to a test khata customer, then a repayment	Balance returns to start
6	Counter-sheet: draft → post (manager); Tables/KOT: open table → order → bill	Stock and revenue update; no 5xx
7	Daily closing preview/submit on the test outlet	Figures equal the bills just rung
8	Accounting: trial balance and Tally export load; totals balance	Debits = credits
9	Supplier invoice upload (small test file); bank-statement preview	Parsed without 5xx
10	Print: queue a bill print for the test branch; print agent picks it up	Job leaves PENDING
11	Payments: live Razorpay test order/webhook; password-reset e-mail delivered	Invoice marked paid; mail received
12	Offline queue: bill while the network is cut, then restore	Bill synced once (idempotent `clientRef`)

Delete or void test data afterwards using the dedicated test outlet; never use real customers' records.

10.2 Integration checks

- The e2e suite (Playwright) runs against staging with the release image; compare with the recorded baseline (G4). Do **not** run it against production or against any server that points at a real database.
- Data checks from §4.6 remain clean; no new `5xx` in the first hour.

10.3 Performance baseline

Record before and after each release on staging (and read-only on prod): bill-create p50/p95, bills-list p95, trial-balance p95, DB CPU, and API CPU at expected peak (<PEAK_BILLS_PER_MIN>). Because a cash bill issues ≈ 24 queries, each ≈ 100–250 ms when the app is far from a hosted database, **place the app in the database's region** and keep pre-checks in one parallel wave. Acceptance: p95 within +20 % of the previous release.

10.4 Sign-off criteria

Criterion	Owner	Signed
Migration applied; `t02` / `t03` clean	DBA	☐
All instances ready; no error-level startup logs	SRE	☐
Smoke tests 1–12 pass (or documented, accepted exceptions)	QA	☐
Performance within threshold; alerts firing correctly (test alert sent)	SRE	☐
Backup taken and restore point recorded	DBA	☐
Release approved to stay live	RM / Eng Lead	☐

11. Rollback Plan

11.1 Triggers (any one)

- Smoke test 3–7 or 12 fails and is not fixable in 20 minutes.
- 5xx rate > 2 % for 5 minutes, or bill-create p95 > 2× baseline, or no bills can be created.
- Data-integrity check (`t02` / `t03`) reports a violation after the release.
- Security finding rated High/Critical introduced by the release.

Decision-maker: RM with the Engineering Lead. Time-box: decide within 15 minutes of the first trigger; prefer rolling back to fixing forward during trading hours.

11.2 Application rollback (default path, ~10 min)

```
kubectl rollout undo deploy/<APP_NAME>                  # or redeploy the previous immutable tag
kubectl rollout status deploy/<APP_NAME>
# restore the previous env set from the vault if it changed; re-run smoke tests 1–5
```

Safe because the schema is additive and backward-compatible with release N-1 (§4.2). Do **not** run the migrator in reverse.

11.3 Database rollback (only if the schema/data change itself is the fault)

#	Step
1	Stop writes: scale the app to 0 (or enable maintenance mode at the LB) and tell outlets to use the offline queue.
2	Prefer the release's tested `*_down.sql` (e.g. `docs/db-optimization/sql/002_integrity_down.sql`, then `001_indexes_down_concurrent.sql`) — no data impact.
3	If data was corrupted: restore to the pre-release snapshot/PITR timestamp into a **new** database, validate with `t01–t03`, then repoint `DATABASE_URL`. **Writes made after the restore point are lost** — export bills/vouchers created since the snapshot first (`created_at > <SNAPSHOT_TIME>`) and re-key or replay them under finance supervision.
4	Deploy the previous image against the restored database; run the smoke suite; re-open traffic.
5	Record exactly what was lost/replayed in the incident ticket.

11.4 Communication plan

When	Audience	Channel	Owner	Message
T-1 day	Outlet owners/managers	E-mail / WhatsApp / in-app	SUP	Window, expected impact, offline-queue reminder, support number
Start / end of window	Internal + outlets	Status page, chat	RM	"Maintenance started / complete"
Rollback decided	Eng lead, ops, outlets	Chat + phone for P1	RM	Reason, ETA, workaround (ring bills on paper / offline queue)
Post-incident	Stakeholders	E-mail	RM	Summary, data impact, next steps within 24 h

12. Post-Deployment Tasks

Task	Detail	Owner	When
Monitoring window	Heightened watch: dashboards open, on-call present; review 5xx, latency, print queue, e-mail/webhook failures; first real **daily closing** of each outlet checked against bills	SRE, SUP	24–72 h
Cleanup	Remove test accounts/data and temporary access; revoke DB admin grants; delete temporary tokens; unfreeze merges; restore DNS TTL	SRE, DBA	Day 1
Backups	Confirm the first scheduled backup after release succeeded; keep the pre-release dump for 30 days	DBA	Day 1–2
Documentation	Update this plan with measured durations, the final env table, runbook corrections; update `AGENTS.md`/README for any new behaviour; archive the evidence	RM, DEV	Week 1
Follow-ups	Close gaps G1–G7 (§1.4); schedule the optional database index/constraint package (§4.7); review unused indexes after a week of real traffic	Eng Lead	Week 1–2
Retrospective	Blameless review: timeline vs plan, what slowed us, alert quality, rollback readiness; actions with owners and dates	RM	≤ 5 working days

Appendix A. Checklist Summary

Phase	Item	Done
Before	All `<PLACEHOLDERS>` filled; contacts confirmed; migrations 0017–0024 committed (G5)	☐
Before	Dockerfile + CI pipeline in place (G1); health/readiness and graceful shutdown decided (G2, G3); test baseline recorded (G4)	☐
Before	Secrets in vault; `JWT_SECRET` ≥ 32 chars; `CORS_ORIGIN` exact; `PAYMENT_PROVIDER=razorpay`; `ALLOW_PROD_SEED` unset	☐
Before	Staging rehearsal with restored prod copy; migration timed; restore tested	☐
Before	DNS/CDN/TLS/firewall configured and verified; alerts and on-call rota live	☐
During	Backup + snapshot verified → migrate → validate → deploy → smoke tests → sign-off	☐
During	Rollback decision made by the time-box; communications sent	☐
After	Monitoring window complete; cleanup; docs updated; retrospective held	☐

Appendix B. Glossary

Term	Meaning
POS	Point of sale — the counter billing screen.
KOT	Kitchen order ticket — bar/restaurant table orders.
Khata	Customer credit ledger (sales on credit and repayments).
Counter sheet	Daily Opening + Inward + Transfer − Closing stock sheet that books unbilled sales.
Syndicate / outlet	A business (tenant) and its shops (branches).
Super Admin	The software provider's platform owner; has no shop screens.
Print agent	Local program at a shop that polls the API and prints to LAN printers.
Drizzle	ORM/migration tool; migrations are SQL files tracked in `drizzle/meta/_journal.json`.
PITR	Point-in-time recovery — restore the database to any moment in a window.
RPO / RTO	Maximum tolerable data loss / time to restore service.
Expand → contract	Add new schema first, remove old in a later release, so old and new code both work.
CONCURRENTLY	Postgres option to build an index without blocking writes; cannot run inside a transaction.
HSTS	Header telling browsers to use HTTPS only.

Appendix C. Contact Matrix

Role	Name	Phone / chat	Backup	Escalation for
Release manager	<NAME>	<PHONE> / <CHAT>	<NAME>	Go/no-go, rollback decision
Engineering lead	<NAME>	<PHONE>	<NAME>	Code defects, scope
On-call SRE	<NAME>	<PAGER>	<NAME>	P1/P2 infrastructure
DBA	<NAME>	<PHONE>	<NAME>	Migration, backup, restore
QA lead	<NAME>	<PHONE>	<NAME>	Smoke/e2e results
Security	<NAME>	<PHONE>	<NAME>	Incidents, secret exposure
Support lead	<NAME>	<PHONE>	<NAME>	Outlet communication
Hosting provider support	<PROVIDER>	<TICKET URL / PHONE>	—	Platform outage
Database provider support	<PROVIDER>	<TICKET URL>	—	Database outage, PITR restore
Payment gateway (Razorpay)	<ACCOUNT MANAGER>	<DASHBOARD / PHONE>	—	Webhook / settlement issues